
vLLM 기초와 추론 속도 최적화

전현준

OT

Agent Harness

Control Plane

Deterministic

Code Mode

Optimization

Local Model

TODAY

vLLM

KV Cache

Route Models

Hybrid Model Arch

Agent Guardrail

Input Guardrail

Output Guardrail

Model Guardrail

Code Guardrail

PJT

Coding Harness

Context Control

목차

01 vLLM 기초

02 성능 지표

03 Attention·KV

04 Batch·Graph·CPU

05 Cache·Quant·Spec

06 분산·최신 Recipe

Chapter 1.

vLLM 기초

vLLM 으로 모델을 서빙할 때는

같은 가중치·같은 샘플링이라도 한 GPU가 동시에 감당하는 요청 수가 달라집니다.

구분	요청 단위 배치 · 연속 공간 예약	반복 단위 배치 · 블록 할당
배치	묶어서 시작해 묶어서 끝남 짧은 요청이 긴 요청을 기다림	매 반복마다 배치를 다시 구성 끝난 자리에 대기 요청이 즉시 진입
KV	문맥 상한만큼 KV 자리를 미리 잡아둠	고정 크기 블록으로 쪼개 지금 필요한 만큼만 할당
빈 공간	padding과 미사용 구간이 계속 자리를 점유	반환된 블록을 재사용하고 공통 prefix는 블록을 공유

모델 품질은 그대로입니다. 바뀌는 것은 같은 VRAM 안의 동시 요청 수입니다.

두 가지 실행 형태

실행 형태가 둘일 뿐, 스케줄러와 KV Cache 관리자는 같은 것을 씁니다.

오프라인 · LLM 클래스

```
from vllm import LLM, SamplingParams

llm = LLM(model="Qwen/Qwen3-4B")
outs = llm.generate(prompts,
    SamplingParams(max_tokens=256))
```

프롬프트 묶음을 넣고 끝까지 돌립니다.
데이터셋 생성·오프라인 평가처럼
일감의 끝이 정해진 작업에 씁니다.

온라인 · vllm serve

```
vllm serve Qwen/Qwen3-4B \
--port 8000
```

```
POST /v1/chat/completions    POST /v1/completions
GET  /v1/models              GET  /metrics
```

OpenAI 호환 HTTP 서버로 상시 대기합니다.
오늘 다루는 최적화는 대부분
이쪽 조건을 전제합니다.

두 경로 모두 같은 EngineCore로 들어갑니다.

최소 실행과 시작 로그

아래 값은 설명용 예시입니다. 버전에 따라 로그 줄 구성이 달라질 수 있습니다.

```
vllm serve Qwen/Qwen3.8-27B --max-model-len 64000 \  
--gpu-memory-utilization 0.85 --max-num-seqs 32 \  
--max-num-batched-tokens 4096 --enable-prefix-caching
```

01

메모리 분할

$\text{total_gpu_memory} \times$
 $\text{gpu_memory_utilization}$
→ weights + runtime + KV

가중치·실행 공간·KV가
실제로 나뉜 비율

02

KV 수용량

GPU KV cache size: 65,536 tokens
Maximum concurrency: 8.00x

동시에 살아 있을 수 있는
토큰의 총량

03

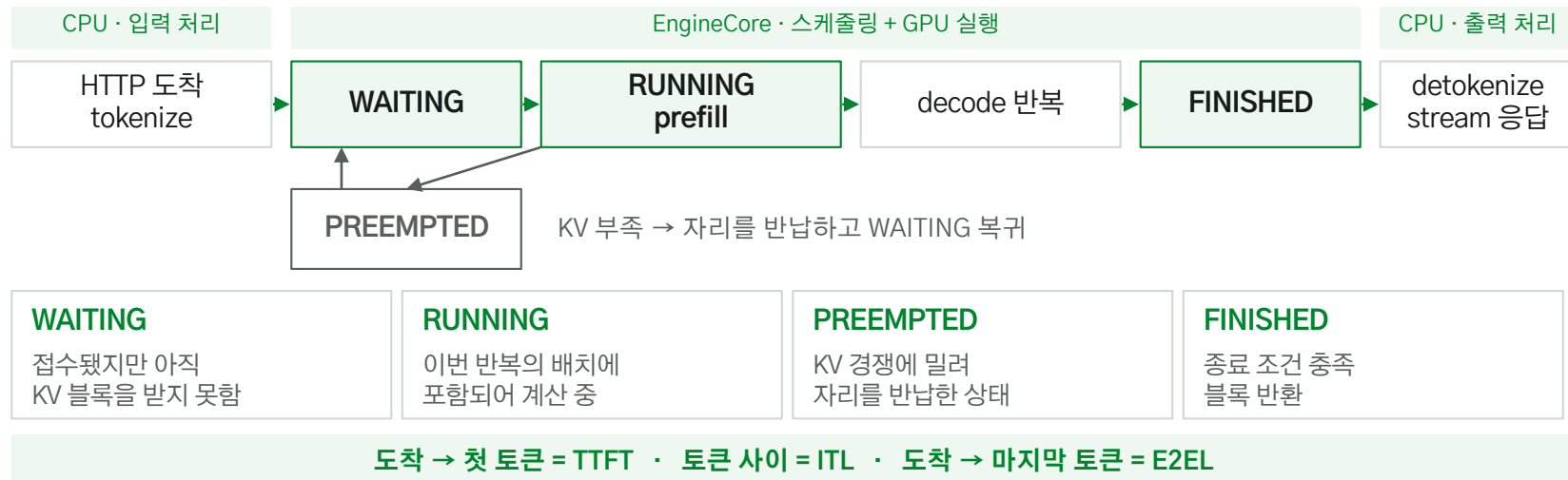
attention backend

Using FLASH_ATTN backend
kv-cache-dtype: auto
(조건에 따라 자동 선택)
쓸 수 있는 KV dtype과
기능 조합이 정해짐

설정 플래그는 요청이고 로그는 응답입니다. 비교 기준은 항상 로그 쪽입니다.

요청 경로와 상태 전이

뒤에 나오는 지표와 튜닝 항목은 모두 이 경로 위의 한 지점을 가리킵니다.

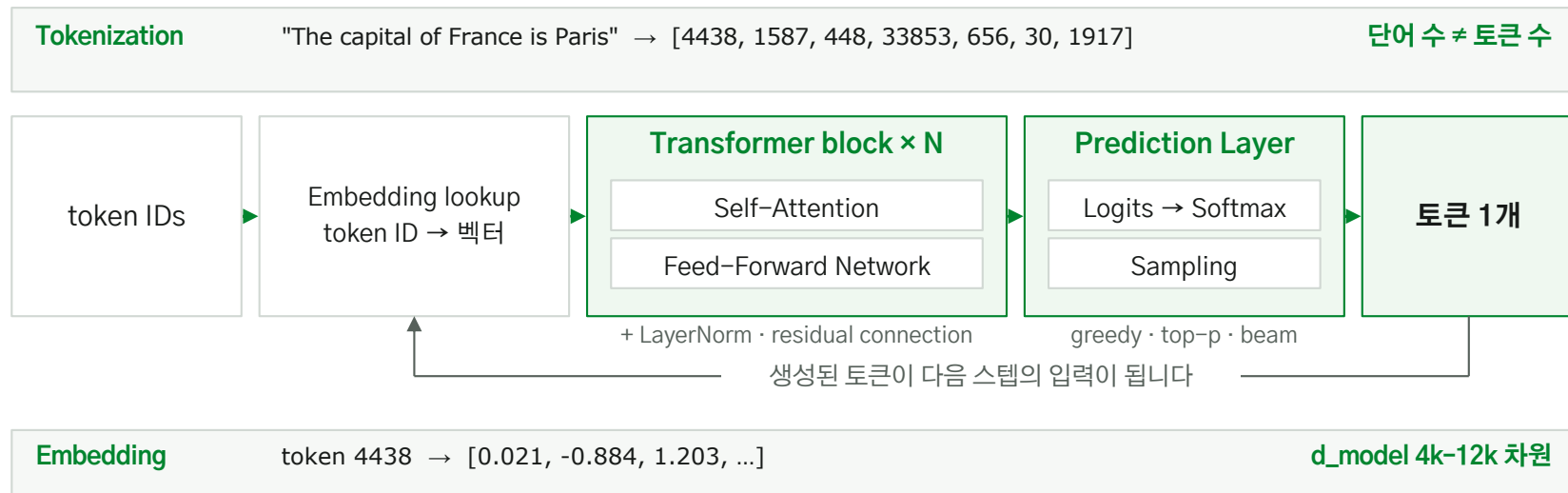


종료 경로: EOS·stop 문자열 / max_tokens 도달 / 클라이언트 취소

TTFT가 나쁘다는 말로는 대기 때문인지 prefill 때문인지 구분되지 않습니다.

토큰 하나가 만들어지는 경로

앞 장이 요청의 생애라면, 이 장은 그 안에서 토큰 하나가 만들어지는 순서입니다.



Transformer block은 layer 수만큼 반복되고, 그림 전체가 토큰마다 다시 돍니다.

샘플링 계수가 실제로 하는 일

Prediction Layer가 만든 확률 분포에서 토큰 1개를 고르는 규칙입니다.

계수	확률 분포에 하는 일	서빙에서 나타나는 영향
temperature	logit을 나눠 분포를 평탄하게 또는 뾰족하게 만듭니다. 0이면 greedy	높을수록 재시도·검증 실패가 늘어 실효 처리량이 내려갑니다
top_p (nucleus)	누적 확률이 p가 될 때까지의 후보만 남깁니다	후보 수가 입력마다 달라져 sampler 지연 편차가 커집니다
top_k	점수 상위 k개 후보만 남깁니다	후보 상한이 고정되어 sampler 비용을 예측할 수 있습니다
repetition / presence penalty	이미 등장한 토큰의 점수를 깎습니다	출력 토큰 수를 바꿔 디코드 단계 수와 KV 점유에 직접 영향
seed	난수 상태를 고정합니다	고정해도 배치 구성이 바뀌면 완전한 재현은 보장되지 않습니다

샘플링은 품질 설정처럼 보이지만, 출력 길이와 후보 수를 통해 서빙 비용에도 관여합니다.

샘플링이 모델 서빙에 주는 영향

샘플링은 GPU 연산량을 거의 늘리지 않지만, 무엇을 얼마나 오래 생성하는지를 바꿉니다.

01

처리량과 지연

출력 토큰 수가 늘면 디코드 단계 수와 KV 점유가 함께 늘어납니다.

길이가 곧 비용

02

재현성과 캐싱

temperature 0과 고정 seed도 배치 구성이 바뀌면 완전 일치를 보장하지 않습니다.

회귀 테스트의 전제

03

검증과 구조화 출력

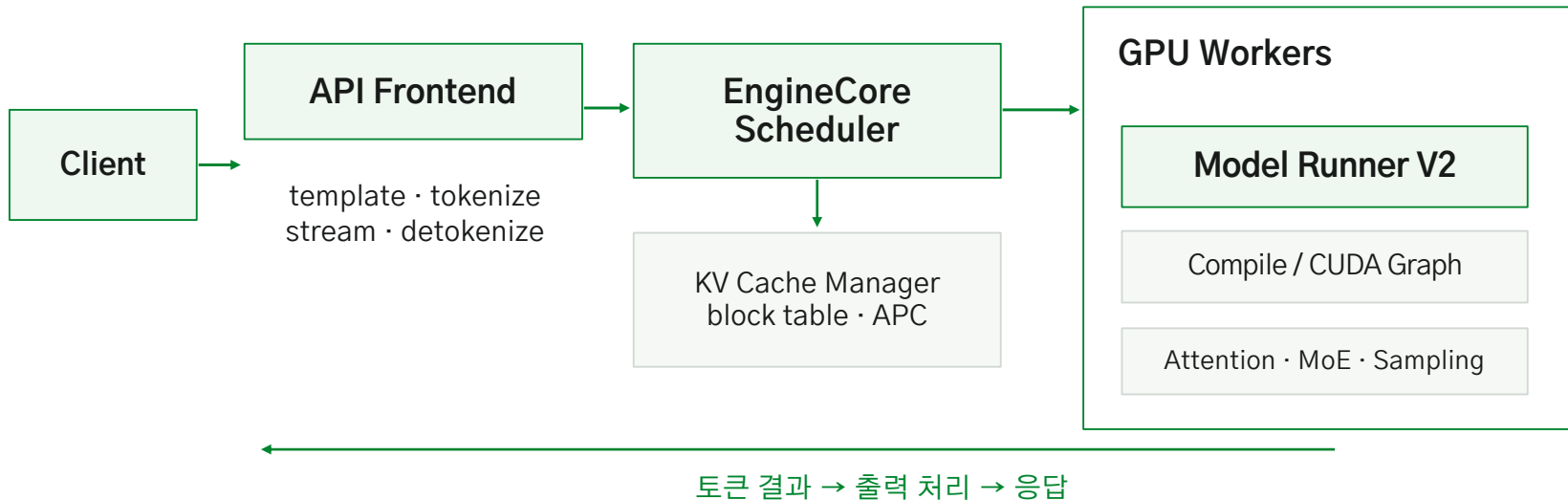
분포를 넓게 열면 JSON 유효성과 도구 인자 성공률, speculative decoding 승인율이 함께 떨어집니다.

재시도 비용

성능을 비교하는 실험에서는 샘플링 설정을 먼저 고정해야 결과를 해석할 수 있습니다.

V1 Engine과 Model Runner V2

Engine 이름의 V1과 Model Runner 버전의 V2를 구분합니다.



TP/DP 등에 따라 프로세스 수와 배치가 달라집니다.

Scheduler는 “누가 얼마나 계산할지”, Runner는 “GPU에서 어떻게 실행할지”를 담당합니다.

Model Runner V2의 개선 지점

모델 계산식만 빨라지는 것이 아니라 입력 준비와 샘플링까지 바꿉니다.

01

GPU-native 준비

지속 배치 상태와 입력 준비를 GPU 친화적으로 구성해 반복적인 CPU 작업과 복사를 줄입니다.

관찰: GPU idle gap

02

Async-first 실행

CPU 준비와 GPU 실행이 가능한 범위에서 겹치도록 설계합니다. 불필요한 동기화를 줄입니다.

관찰: host/device timeline

03

샘플링·메모리 개선

Triton 기반 sampler와 v0.29의 batch-sharded sampling, graph memory profiling이 추가되었습니다.

관찰: sampler·logits·HBM

용어 정리

약어는 이 장에서 한 번만 풀고, 이후 슬라이드에서는 약어로만 씁니다.

지표 용어	뜻	실행 용어	뜻
TTFT Time To First Token	요청 시작부터 첫 출력 토큰까지	Prefill	입력 전체의 K·V를 한 번에 계산하는 단계
TPOT Time Per Output Token	첫 토큰 이후 토큰 하나당 평균 시간	Decode	토큰을 하나씩 생성하는 반복 단계
ITL Inter-Token Latency	연속한 두 출력 토큰 사이의 간격	KV Cache Key-Value Cache	과거 K·V를 보관해 재계산을 없애는 메모리
E2EL End-to-End Latency	요청 시작부터 마지막 토큰까지	Block	KV Cache를 나눈 고정 크기 단위
Throughput	초당 출력 토큰 수 초당 완료 요청 수	Preemption	KV 부족으로 자리를 반납하고 대기로 복귀
Goodput	품질·지연 조건을 통과한 요청만 센 완료 수	APC Automatic Prefix Caching	이미 계산한 공통 prefix의 KV를 재사용
SLO Service Level Objective	서비스가 지키기로 한 지연·품질 기준	TP · PP · DP Tensor·Pipeline·Data	가중치·레이어·요청 중 무엇을 나눌지

TPOT는 평균, ITL은 간격의 분포입니다. 평균이 좋아도 p99는 나쁠 수 있습니다.

Chapter 2.

LLM Model Serve 성능 지표

요청이 빠르다

단일 요청의 체감 속도와 서버 전체 처리량은 같은 숫자가 아닙니다.

01

첫 토큰이 빨리

TTFT가 짧아야 질문 후 기다리는 시간이 줄어듭니다. Queue와 prefill이 함께 영향을 줍니다.

대화·RAG 첫 반응

02

생성이 끝가지 않게

TPOT와 ITL을 봅니다. 평균이 좋아도 p99가 나쁘면 스트리밍이 중간에 멈춥니다.

긴 답변·추론 모델

03

더 많은 요청을 처리

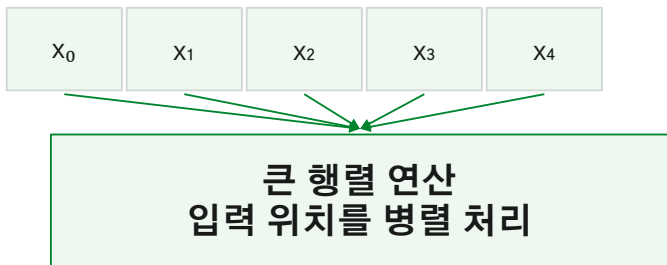
Output tokens/s와 requests/s를 봅니다. SLO를 지킨 요청만 세는 goodput을 함께 측정합니다.

동시 사용자·GPU 비용

Prefill과 Decode

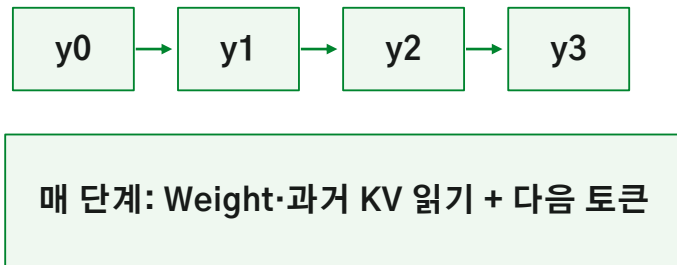
배치·문맥 길이·모델 구조에 따라 병목은 달라집니다.

Prefill · 입력을 한 번에



길이가 긴 입력에서는 compute와 attention I/O가 중요합니다.

Decode · 출력을 순차적으로



작은 배치의 긴 생성에서는 HBM 대역폭과 launch 비용이 중요할 수 있습니다.

Prefill 최적화가 긴 답변의 decode 시간을 자동으로 줄이지는 않습니다.

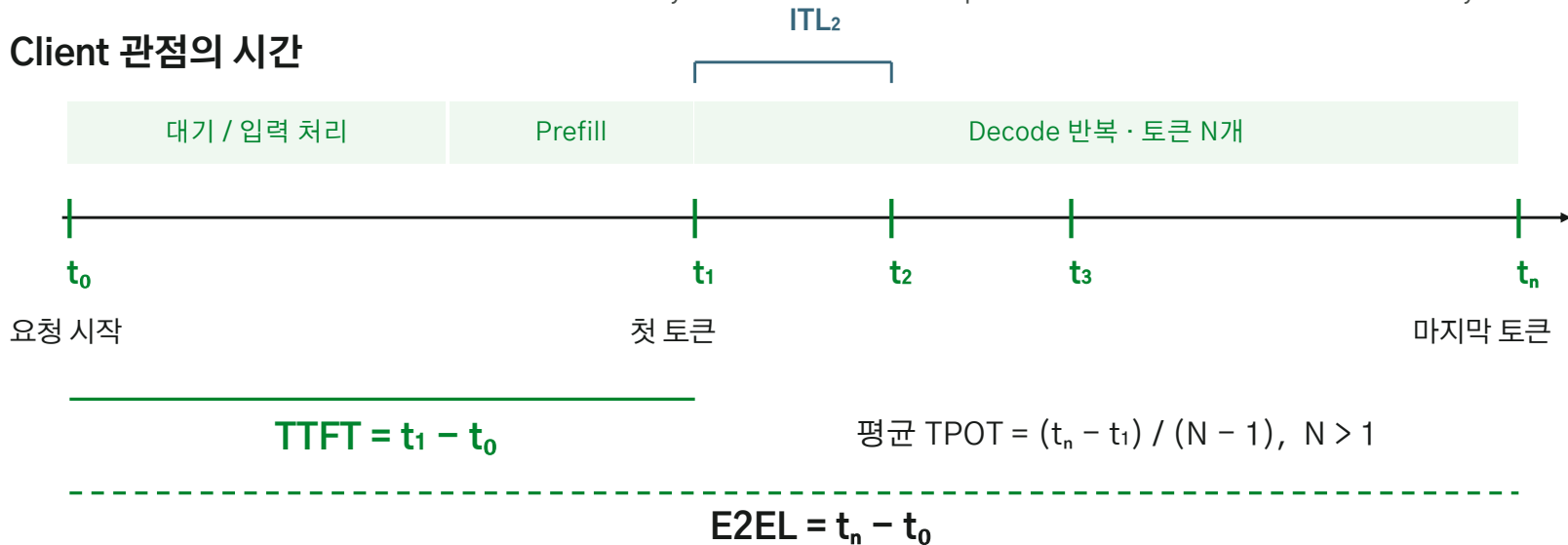
지표의 시간 축

t_0 : 요청 시작, t_1 : 첫 출력 토큰, t_n : 마지막 출력 토큰

TTFT는 대기·prefill을, TPOT은 decode 구간의 평균 간격을, E2EL은 전체를 나타냅니다.

TTFT Time To First Token · ITL Inter-Token Latency · TPOT Time Per Output Token · E2EL End-to-End Latency

Client 관점의 시간



스트림 청크가 여러 토큰이면 청크 간격을 토큰별 ITL로 그대로 해석하지 않습니다.

프로파일별 병목(샘플)

시험 프로파일	입력 / 출력 토큰	우선 의심할 비용	핵심 지표
짧은 대화	1,024 / 256	CPU·tokenizer·launch	TTFT · p99
긴 생성	1,024 / 3,072	반복 decode·HBM	TPOT · ITL
RAG 문서	8,192 / 1,024	prefill·queue	TTFT · goodput
장문 + 동시 요청	32,768 / 1,024	KV 용량·대역폭	preemption · KV
공통 prefix 반복	8,192 공통 + 512 / 1,024	중복 prefill	cache hit · TTFT

출력 길이는 추론형 모델의 장문 응답을 가정한 값이며, 측정 시 --ignore-eos를 함께 지정합니다. 출처: vLLM Benchmark CLI 문서.

같은 모델도 프로파일이 바뀌면 최적 설정이 바뀝니다.

CLI 파라미터 한눈에 보기

같은 파라미터가 목표에 따라 반대 방향으로 움직입니다.

파라미터	처리량을 올릴 때	지연을 줄일 때
<code>--max-num-seqs</code>	올림 — 동시 요청 수 확대	내림 — 배치 경쟁 완화
<code>--max-num-batched-tokens</code>	올림 — 한 회차 진전 확대	내림 — decode 간섭 축소
<code>--gpu-memory-utilization</code>	올림 — KV pool 확대	여유 확보를 우선
<code>--enable-prefix-caching</code>	컴 — 반복 prefill 제거	컴 — TTFT 단축
TP / replica	replica 수를 늘림	TP 크기를 늘림
Weight quantization	컴 — 메모리 여유 확보	조건부 — 항상 빠르지는 않음
Speculative decoding	보통 불리	낮은 부하에서 유리

두 목표를 동시에 만족시키는 설정은 없습니다.

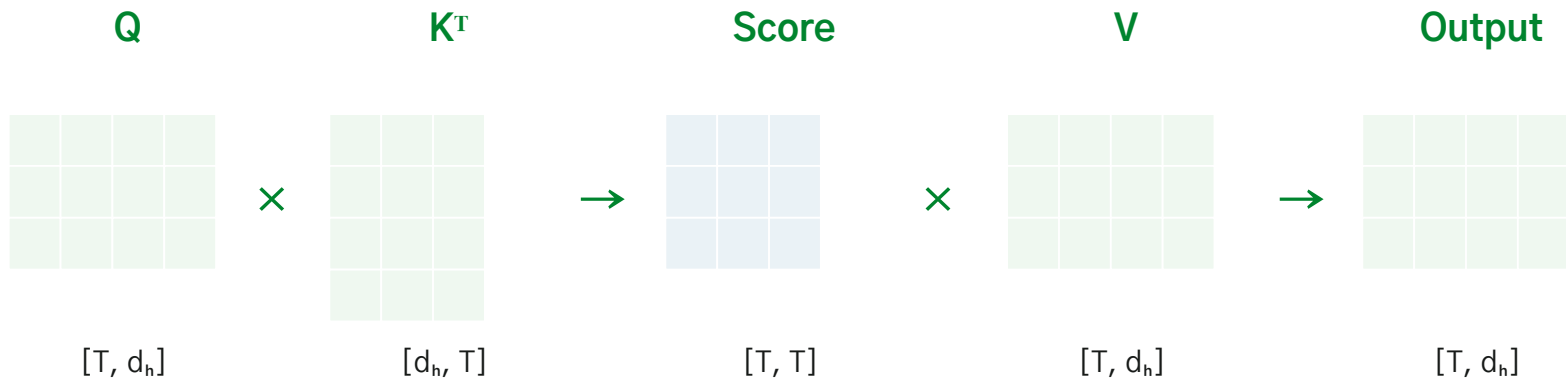
Chapter 3.

Attention · KV Cache · 블록 관리

메모리와 연산, 실제 모델 서빙의 병목이 발생하는 원리를 이해합니다.

Attention 연산 구조

원본의 행렬 관점은 유지하되 attention head 기준으로 차원을 명시합니다.



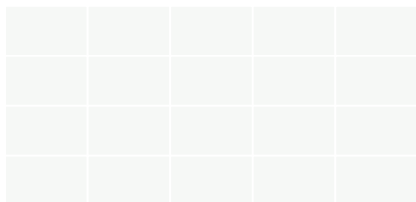
$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_h}) V$$

K와 V는 과거 토큰의 조회 자료입니다. Decode 때 이미 계산한 값을 다시 만들 필요가 없습니다.

KV Cache의 역할

아래 복잡도는 Dense attention 부분만의 비교입니다. 전체 LLM 연산량과 다릅니다.

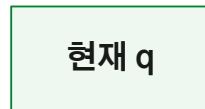
과거 K·V를 매번 다시 계산



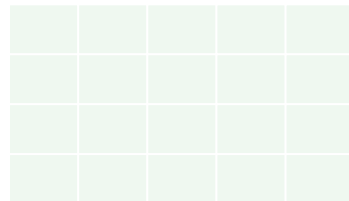
전체 prefix를 재실행
과거 계산도 반복

$$O(T^2 d_h)$$

Cache: 새 q와 과거 KV만 읽기



×



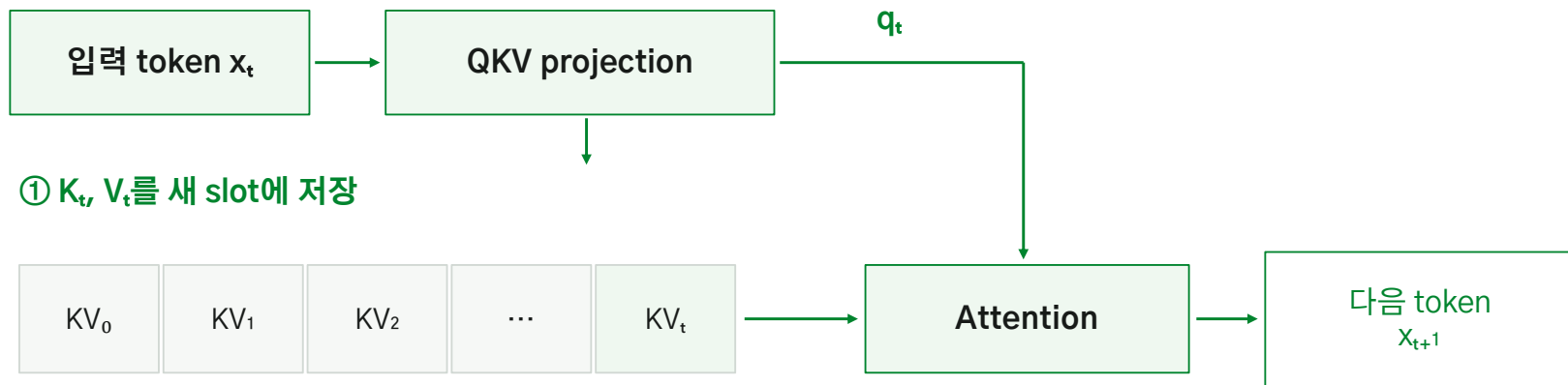
Cache의 K^T

$$O(T d_h)$$

Cached decode의 attention은 $O(T \cdot d_h)$. 전체 추론이 항상 memory-bound라는 뜻은 아닙니다.

토큰 생성 시 Cache 변화

현재 토큰의 K·V를 append하고, 필요한 과거 KV를 읽어 attention을 계산합니다.



② q_t 가 기존 + 현재 KV를 조회

projection은 token 벡터에 학습된 가중치 $W_Q \cdot W_K \cdot W_V$ 를 곱해 Q·K·V를 만드는 선형 변환입니다.
과거 K·V의 변환 결과를 재사용하고, 현재 token의 변환 자체는 매 step 수행됩니다.

KV Cache는 계산을 절약하지만, 문맥 길이와 동시 요청 수에 비례해 커집니다.

KV 메모리 공식

MHA·GQA·MQA를 같은 공식으로 설명합니다. MLA·recurrent state에는 별도 모델이 필요합니다.

$$M_{kv} \approx 2 \times B \times L \times T \times n_{kv} \times d_h \times b$$

B

동시에 활성인 sequence 수

L

KV를 저장하는 layer 수

T

sequence당 cached token 수

 n_{kv}

Key·Value head 수

 d_h

head dimension

b

원소당 bytes · BF16=2

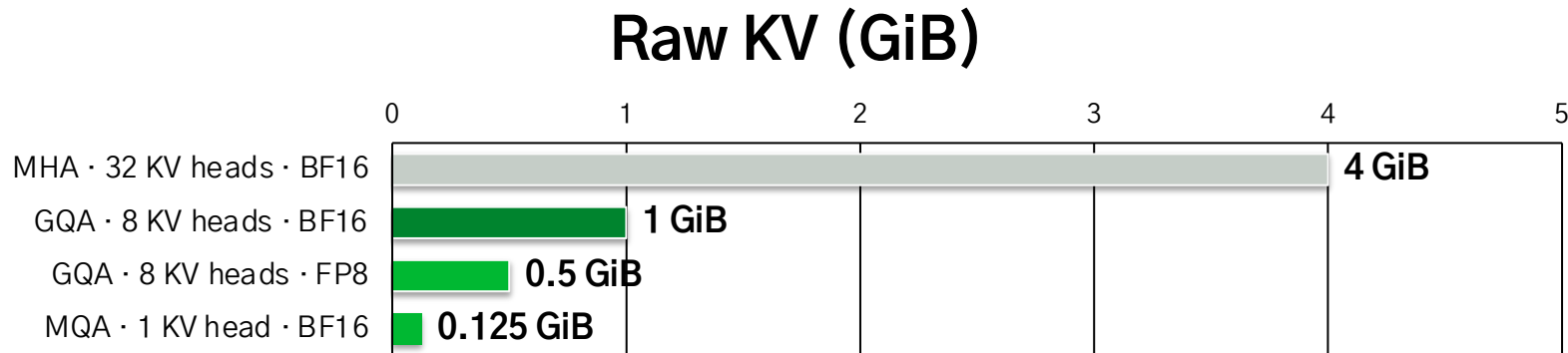
n_{kv} 는 attention 방식에 따라 다릅니다 — MHA는 Q head 수와 동일, GQA는 여러 Q head가 공유, MQA는 1개.
요청별 길이가 다르면 $B \times T$ 대신 각 요청의 T_i 를 합산합니다. 2는 K와 V 두 개를 뜻합니다.

원본의 hidden_size 식은 MHA 사례로 한정합니다. 실제 config의 num_key_value_heads를 확인합니다.

MHA · GQA · MQA의 KV 크기

계산 예시: $L=32$, $T=8,192$, $d_h=128$, $B=1$. Scale·정렬·metadata는 제외합니다.

공식에 대입한 계산 결과



B=16개의 동일 GQA/BF16 요청이면 raw KV 합계는 16 GiB입니다.

MHA 32 heads BF16 = 4 GiB → GQA 8 heads = 1 GiB → FP8 = 0.5 GiB → MQA 1 head = 0.125 GiB.

GPU 메모리 구성 요소

Graph capture·activation·workspace·통신 버퍼·adapter·draft에도 공간이 필요합니다.

GPU HBM 예산



개념도 · 블록 크기는 실제 메모리 비율이 아닙니다.

자동 예약도 공짜 공간이 아닙니다

graph capture와 workspace가 늘면 같은 GPU에서도 KV에 남는 공간이 달라집니다.

메모리 부족과 연산 포화를 구분합니다

KV가 꽉 찬 상태의 재계산 문제와 compute가 포화된 문제는 다른 설정으로 해결합니다.

gpu-memory-utilization은 연산 사용률이 아니라 메모리 예산 비율입니다. OOM 여유를 남깁니다.

연속 예약의 한계

Dynamic/Static cache는 전형적인 비교 사례

Static / 연속 예약



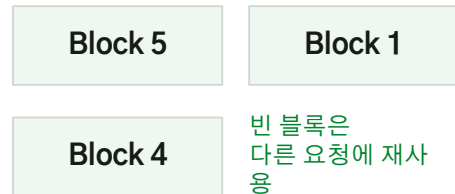
최대 길이를 예약하면 실제로 쓰지 않는 구간이 생깁니다.

Dynamic 확장 사례



concat 기반 cache의 사례입니다. 다른 dynamic cache 구현은 다를 수 있습니다.

Paged KV



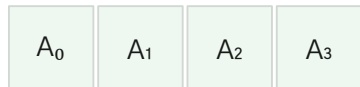
요청별 긴 연속 공간 대신 고정 크기 block을 할당합니다.

논리 블록과 물리 블록

교육용 block size=4, 모든 block ID는 0부터 시작합니다.

요청 A · 논리 블록

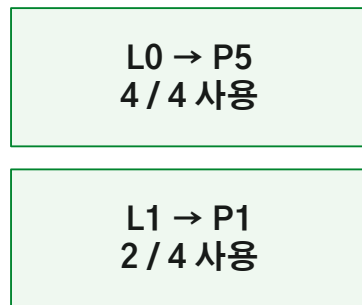
L0



L1



Block Table



GPU · 물리 블록

P0



P1



P2



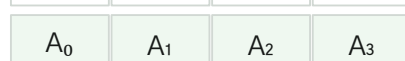
P3



P4



P5



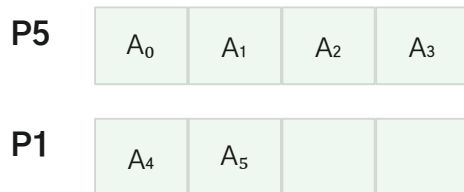
Block size=4 · P0...P5 · L0...L1

요청의 토큰 순서는 논리적으로 연속적입니다. 물리 KV는 흩어져 있어도 block table로 찾아갑니다.

블록 증가 과정

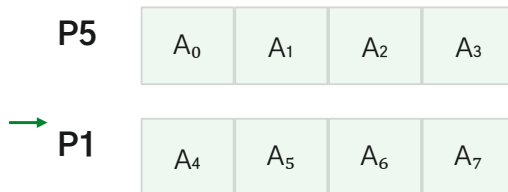
6 → 8 → 9개 토큰으로 늘어나는 동일 요청을 추적합니다.

6 tokens



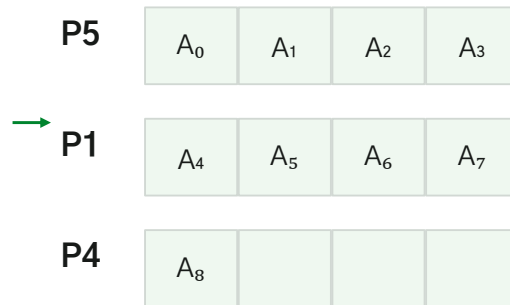
마지막 block에 2칸 여유

8 tokens



남은 두 slot을 채움

9 tokens

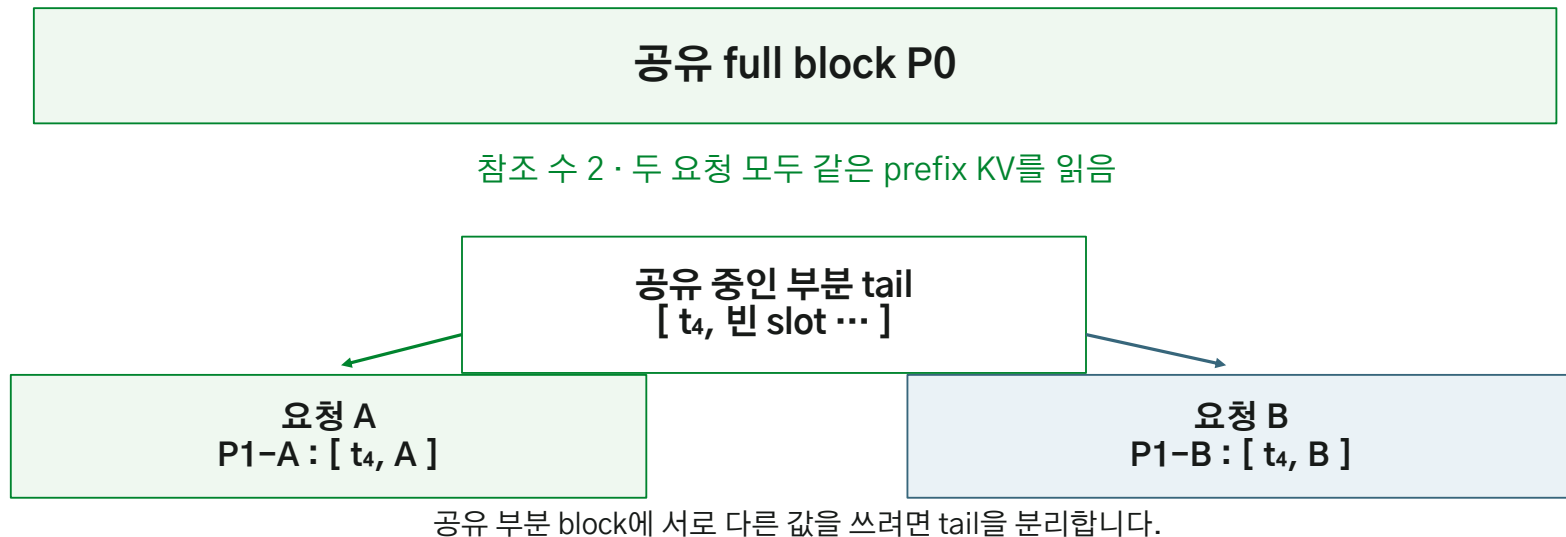


새 P4 할당 · 3칸 여유

단순 full-attention 예시에서 남은 공간은 마지막 블록에 한정됩니다. 큰 연속 재할당이 필요 없습니다.

블록 공유와 Copy-on-Write

Parallel sampling과 공통 prefix의 출발점: block 공유와 Copy-on-Write



같은 KV를 가리키는 것과, 서로 다른 이후 토큰까지 동일하게 처리하는 것은 다릅니다.

블록 단위 Softmax 처리

“블록별 softmax 후 단순 평균”으로 바꾸면 원래 attention이 아닙니다.

KV Block 0

부분 최대 m_j
정규화 합 $z_j \cdot$ 가중합 u_j

KV Block 1

부분 최대 m_j
정규화 합 $z_j \cdot$ 가중합 u_j

KV Block 2

부분 최대 m_j
정규화 합 $z_j \cdot$ 가중합 u_j


$$O = \frac{\sum_j \exp(m_j - m) \cdot u_j}{\sum_j \exp(m_j - m) \cdot z_j} \quad m = \max_j m_j$$

m_j : block별 score 최대, z_j : $\exp(\text{score} - m_j)$ 의 합, u_j : 같은 가중치로 V 를 합친 값

블록의 부분 통계와 출력을 올바르게 결합해야 전체 attention과 같은 결과를 얻습니다.

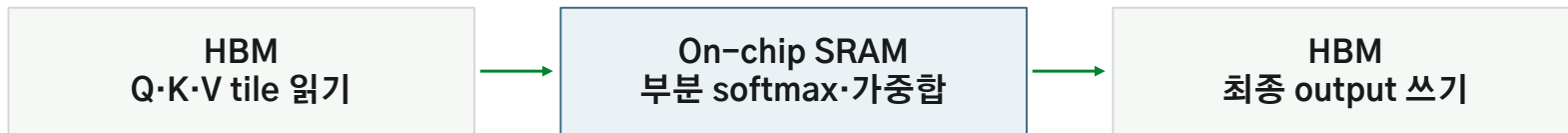
Paged KV와 FlashAttention

하나는 저장 위치를 관리하고, 다른 하나는 attention 계산 중 메모리 왕복을 줄입니다.

① 저장 관리 계층



② 연산 I/O 계층



T×T score 전체를 HBM에 저장하지 않고, tile과 부분 통계를 결합합니다.

Attention Backend 선택

자동 선택 → 실제 선택 로그 확인 → 필요한 경우에만 강제 지정

조건	우선 확인할 경로	추가 검증
MHA / GQA · Ampere·Hopper	FLASH_ATTN 우선 후보	dtype · head size · KV dtype
MHA / GQA · Blackwell	FLASHINFER 우선 후보	kernel 모드 · graph 호환성
MLA / Sparse MLA	MLA 전용 backend	sparse 패턴 · DCP · GPU 세대
Hybrid / multimodal	layer·KV group별 지원	SWA · recurrent state · encoder

Chapter 4.

Batch · CUDA Graph · CPU 최적화

Continuous Batching

고정된 Request 기반의 batch 사례입니다.

고정 request batch · 예시

	1	2	3	4	5	6	7	8
Slot 0	A	A	A	A				
Slot 1	B	B						
Slot 2	C	C	C	C	C	C		

iteration →

새 요청 D는 batch 종료를 기다림

Continuous Batching

	1	2	3	4	5	6	7	8
Slot 0	A	A	A	A	F	F	F	F
Slot 1	B	B	D	D	D	E	E	E
Slot 2	C	C	C	C	C	C	G	G

iteration →

B 종료 후 D, A 종료 후 F를 투입

완료된 요청을 빼고 새 요청을 넣어 빈 자리를 활용합니다. 요청별 응답 반환과 배치 자원 회수도 구분합니다.

Packed Token과 Slot Mapping

Padding을 줄여도 요청 경계를 잃으면 안 됩니다.

A: 논리 길이 6
물리 block [5, 1]

B: 논리 길이 2
물리 block [2]

input token

A_0

A_1

A_2

A_3

A_4

A_5

B_0

B_1

position

0

1

2

3

4

5

0

1

slot mapping

20

21

22

23

4

5

8

9

요청 A의 시작·끝: [0, 6) | 요청 B: [6, 8) | block size = 4

slot = physical_block_id × block_size + offset. Hidden states는 [총 토큰 수, hidden_dim] 등으로 처리됩니다.

Chunked Prefill

Decode를 먼저 배치하고 남은 예산에 prefill chunk를 넣는 기본 정책을 설명합니다.

개념 예제: budget 12 = decode 4 + prefill 8

긴 입력 P의 남은 20 tokens를 8 + 8 + 4로 나눕니다.

Iteration 1	D	D	D	D	P0	P1	P2	P3	P4	P5	P6	P7
Iteration 2	D	D	D	D	P8	P9	P10	P11	P12	P13	P14	P15
Iteration 3	D	D	D	D	P16	P17	P18	P19				

D: 진행 중인 decode 1 token씩

P: 긴 입력의 prefill chunk

max_num_batched_tokens는 한 iteration의 토큰 예산이지, 요청 하나의 최대 context 길이가 아닙니다.

Token Budget의 트레이드오프

작은 예산은 decode 간섭을 줄이고, 큰 예산은 prefill을 더 많이 처리할 수 있습니다.

1 고정할 조건

모델·GPU·요청 집합·동시성·출력 길
이는 그대로 둡니다.

2 바꿀 변수

max_num_batched_tokens만 2K →
8K → 16K로 바꿉니다.

3 동시에 볼 결과

TTFT·ITL·output tok/s·preemption.
숫자는 최적값이 아닌 시험점입니다.

실행 예시 / GPU 실측은 별도

```
vllm serve "$MODEL" \  
  --max-num-batched-tokens 2048 \  
  --max-num-seqs 64 \  
  -O2
```

다음 실험: 2048만 변경
8192, 16384 순서로 비교
max-num-seqs는 이후 별도 sweep

두 Config 을 동시에 바꾸면 개선 원인을 분리하기 어렵습니다.

KV 부족과 Preemption

높은 동시성이 항상 높은 처리량으로 이어지지 않습니다.



Preemption = KV가 부족하면 진행 중인 요청을 중단해 KV를 회수하고, 여유가 생기면 다시 스케줄해 재계산하는 동작입니다.

1 증상

KV usage 상승, preemption 증가, TTFT/E2E 꼬리 지연이 함께 커질 수 있습니다.

2 검증할 변경

max_num_seqs·context budget 축소, KV 여유 확보, calibrated FP8 또는 적정 TP를 비교합니다.

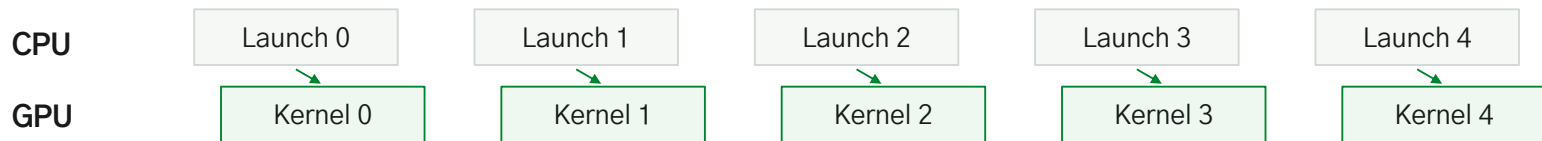
Preemption이 반복되면 active sequences·문맥 길이·KV 예산부터 확인합니다. 메모리 비율만 무작정 올리지는 않습니다.

CUDA Graph

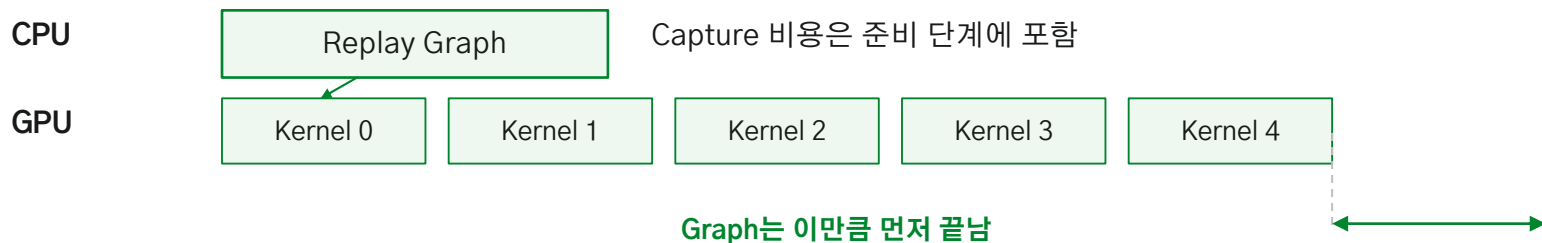
Eager on/off 는 launch 비용과 capture/replay 로 설명할 수 있음

개념적인 timeline

Eager



Graph Replay



Graph는 모델의 토큰 의존성을 없애지 않습니다. 반복 실행 경로를 효율적으로 재생합니다.

Compile과 CUDA Graph

Compile은 연산 구성·fusion을 최적화하고, Graph는 실행 제출 비용을 줄입니다.

설정	주요 목적	사용할 때
-O 0	최적화를 최소화	시작 시간·진단 대조군
-O 1	빠른 compile + PIECEWISE	개발 반복·짧은 운영 수명
-O 2 · 기본	더 넓은 최적화 + FULL/PIECEWISE	steady-state 기준선
-O 3	현재 문서상 O2와 동일	상위 숫자 = 더 빠름으로 가정 금지
--enforce-eager	compile·CUDA Graph 비활성화	문제 격리·시작 비용 비교

Eager는 OOM을 “방지하는 모드”가 아닙니다. Graph 메모리를 줄일 수 있어도 다른 OOM 원인은 남습니다.

Performance Mode 선택

런타임 실행 경로를 latency 지향과 throughput 지향 중에서 한 값으로 선택합니다.

01

balanced · 기본

특정 부하 목표로 치우치지 않은
기준선입니다. 먼저 이 설정으로
지표를 확보합니다.

`--performance-mode balanced`

02

interactivity

작은 배치에서 요청별 지연을 우선
하는 kernel·graph 경로를 지향
합니다.

`--performance-mode interactivity`

03

throughput

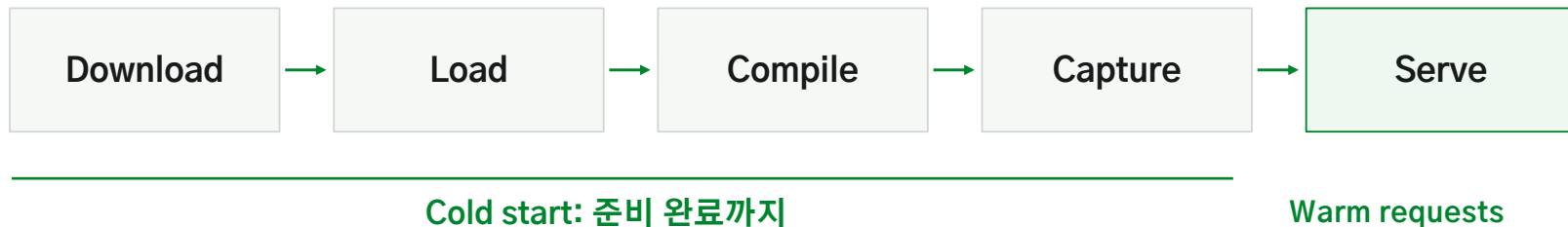
높은 동시성에서 총 token 처리량
을 우선하는 실행 경로를 지향합니
다.

`--performance-mode throughput`

Mode 이름은 성능 보장이 아닙니다. 실제 graph·kernel과 SLO 내 goodput을 확인합니다.

Cold Start와 Warm Serving

가중치 다운로드, 로딩, JIT, graph capture를 token 생성 시간과 섞지 않습니다.



구분해서 저장

최초 시작, cache가 있는 재시작, warm serving의 지표를 서로 다른 결과 파일에 남깁니다.

Cache miss를 확인

Compile cache key가 달라지면 다시 compile할 수 있습니다. 같은 모델 이름만으로 재사용을 가정하지 않습니다.

Compile cache는 모델·설정·GPU·소프트웨어 조합에 의존합니다. 같은 조건인지 확인한 뒤 재사용합니다.

CPU 경로의 병목

Tokenization·입력 처리·스케줄러·detokenization·네트워크도 추론 경로입니다.

1 작은 요청이 많을 때

GPU utilization이 낮는데 TTFT가 커지면 CPU 코어와 tokenizer 시간을 봅니다.

2 멀티소켓 서버

GPU 가까운 NUMA node에 worker를 배치하는 실험을 합니다.

3 성공 조건

CPU 병목이 줄고 TTFT 또는 GPU 유휴 구간이 개선되어야 합니다.

실행 예시 / GPU 실측은 별도

```
# tokenizer-bound 후보  
VLLM_USE_FASTTOKENS=1 \  
vllm serve "$MODEL" -O2
```

```
# 멀티소켓·멀티 GPU 후보  
vllm serve "$MODEL" \  
  --tensor-parallel-size 4 \  
  --numa-bind -O2
```

Chapter 5.

Cache · Quantization · Speculation

재사용·압축·검증의 이득과 비용을 비교합니다.

Automatic Prefix Caching

KV 캐시는 맨 앞 블록부터 연속으로 일치하는 구간까지만 재사용됩니다.

요청 1	공통 P0	공통 P1	공통 P2	고유 S1	캐시 저장
요청 2	공통 P0	공통 P1	공통 P2	고유 S2	앞 3블록 HIT
요청 3	다른 P0	공통 문장	공통 문장	고유 S3	전체 MISS

요청 2 — 앞 3블록이 요청 1과 같음 → P0·P1·P2의 KV를 그대로 가져오고 고유 S2만 새로 계산

요청 3 — 첫 블록부터 다름(다른 P0) → 뒤 블록의 hash도 모두 달라져, 중간 문장이 같아도 재사용 불가

cache hit는 prefix가 처음부터 같을 때만 발생하며, 중간에 같은 문장이 있어도 해당되지 않습니다.

캐시 친화적 프롬프트 순서

고정된 내용을 앞에, 매번 바뀌는 내용을 뒤에 두면 prefix가 안정됩니다.

재사용에 불리한 순서

● 고정 ● 변동 ● 질문

매번 바뀌는 값

고정 지시문

고정 자료

질문

매번 바뀌는 값이 맨 앞 → 뒤의 고정 블록까지 hash가 달라져 전부 재계산

의미를 바꾸지 않고 순서만 조정

고정 지시문

고정 자료

변동 정보

질문

고정 블록이 맨 앞 → 앞 2블록은 매 요청 재사용, 변동 정보와 질문만 계산

순서 조정은 의미가 같은 범위에서만 합니다. Prefix Caching 는 prefill만 줄일뿐 출력 생성 시간은 그대로입니다.

Weight와 KV Quantization

“INT4 모델”이라고 해서 KV Cache도 INT4라는 뜻은 아닙니다.

KV dtype	Weight · BF16	Weight · INT4 계열
BF16 KV	Weight BF16 KV BF16	Weight INT4 KV BF16
FP8 KV	Weight BF16 KV FP8	Weight INT4 KV FP8

개념 분리표 · 모든 model/kernel에서 네 가지 조합이 지원된다는 뜻은 아닙니다.

실험 manifest에는 weight format과 KV dtype, 실제 kernel/backend를 따로 기록합니다.

FP8 KV Cache

BF16 2 bytes → FP8 1 byte. Raw KV payload 기준 절반이며 전체 메모리 절반은 아닙니다.

1 대표 입력으로 calibration

언어·업무·길이 분포를 대표하는 데이터로 Q/K/V scale을 준비합니다.

2 기본형과 비교

같은 요청에서 native KV와 calibrated FP8 KV의 품질·길이별 성능을 비교합니다.

3 최근 변경

v0.28에서 runtime calculate_kv_scales가 제거됐습니다. 옛 예제를 그대로 쓰지 않습니다.

실행 예시 / GPU 실측은 별도

```
vllm serve "$CALIBRATED_MODEL" \  
  --kv-cache-dtype fp8 \  
  --enable-prefix-caching \  
  -O2
```

```
# 지원 모델·backend에 한해  
# 민감한 layer를 native로 유지  
# --kv-cache-dtype-skip-layers \  
#   sliding_window
```

Per-head scale 등은 backend 조건이 있습니다. FP8 저장 지원과 FP8 연산 가속 지원도 구분합니다.

Weight 축소와 Decode 속도

메모리 절감·연산 경로·dequantization 비용이 함께 바뀌므로 더 빨라진다고 단정할 수 없습니다.

후보	줄이려는 비용	확인할 조건
BF16 / FP16	비교 기준선	충분한 Weight + KV 공간
AWQ / GPTQ 계열 INT4	Weight 크기·읽기	checkpoint format · Marlin 등 kernel
FP8 W8A8	Weight·activation 및 matmul	GPU의 native FP8 경로
NVFP4 / MXFP4	더 낮은 정밀도 저장·연산	서로 다른 format · GPU별 지원
BitsAndBytes	기존 실험의 비교 대상	v0.28부터 out-of-tree plugin 경로

새 정밀도의 품질 통과와 SLO 내 tok/s/GPU 개선을 모두 확인한 뒤 채택합니다.

Quality Gate

출력이 짧아져 생긴 가짜 속도 향상과 실제 추론 최적화를 구분합니다.

01

정답·업무 품질

같은 Golden Set의 정확도·근거성·도구 선택 성공률을 비교합니다. 허용 회귀를 먼저 정합니다.

품질 기준선

02

긴 문맥의 품질

찾아야 할 정보가 앞·중간·끝에 있을 때를 나누고, 문맥이 길어질 때의 회귀를 봅니다.

KV 양자화 검증

03

출력 행동의 변화

출력 토큰 수·종료 이유·반복·JSON 유효성·reasoning budget을 함께 기록합니다.

길이·샘플링 통제

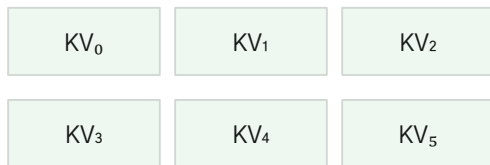
Thinking을 끄거나 max_tokens를 줄이는 것은 별도 제품 정책 변경입니다. 엔진 가속과 같은 실험으로 합치지 않습니다.

Hybrid Attention 구조

Full attention·sliding window·linear/recurrent state를 섞는 hybrid 구조

같은 모델 안에서도 cache의 성장 방식이 다릅니다.

Full attention



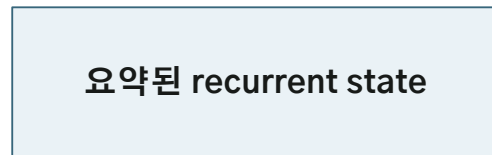
과거 token 수에 따라 증가

Sliding window



window 밖 token은 attention 범위에서 제외

Linear / recurrent



길이와 다른 방식으로 state를 관리

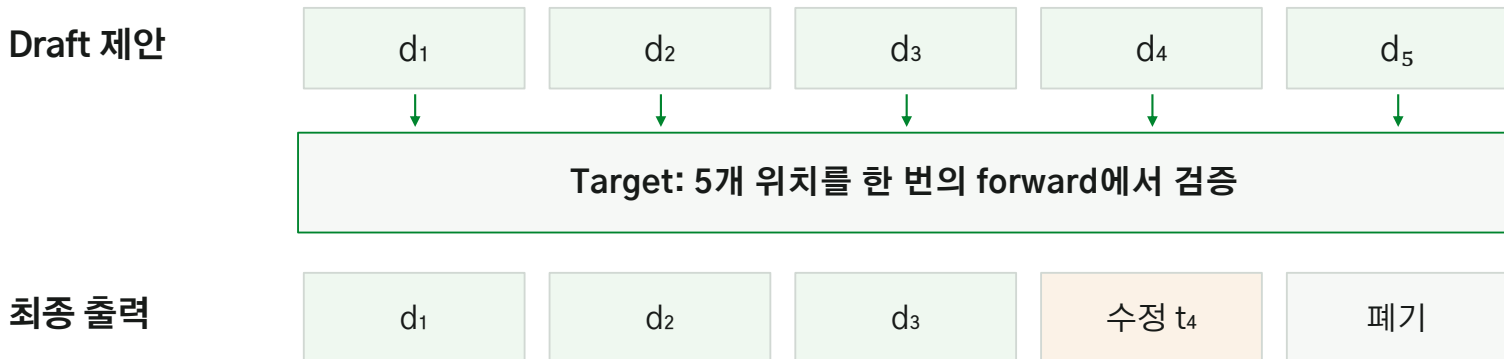
Hybrid cache manager는 종류별 state와 할당 단위를 함께 다룹니다.

원본의 $B \times L \times T$ 식을 모든 최신 모델에 그대로 대입하지 않습니다. Layer별 cache 종류와 layout을 확인합니다.

Speculative Decoding 흐름

원본의 “쉬운 토큰은 작은 모델도 맞힌다”를 실제 승인·폐기 흐름으로 연결합니다.

Greedy 예시 · 앞 3개가 맞고 4번째가 불일치



Greedy: 토큰 일치 여부 | Sampling: 승인 확률과 보정 분포를 사용하는 검증

Target은 K개 위치를 한 번의 verification forward로 평가합니다. 비용이 보통 1-token decode와 같지는 않습니다.

Acceptance와 비용 모델

아래 식은 보정/보너스 토큰 1개를 포함한 단순화된 비용 모델입니다.

$$\text{cost/token} \approx \frac{C_{\text{draft}} + C_{\text{verify}} + C_{\text{overhead}}}{E[A] + 1}$$

분자: 새로 드는 비용

draft 실행 + target 검증 +
scheduling·통신·padding

낮을수록 유리

분모: 실제 출력한 토큰

A는 승인된 draft token 수. 제안 길이
K와 다릅니다.

승인 길이를 측정

비교 대상: 일반 decode

이 값이 기존 1-token decode 비용보
다 작아야 이득입니다.

같은 QPS에서 비교

평균 승인 길이만 보지 말고 실제 QPS에서 TPOT·ITL·throughput과 draft/verify 시간을 함께 봅니다.

Speculation 방식의 종류

지원 조합은 모델·runner·attention backend·병렬화 설정에 따라 달라집니다.

방식	Draft를 만드는 방법	선택 조건
N-gram / suffix	입력·출력의 반복 패턴 활용	별도 모델 없이 시작
Draft model	별도의 작은 모델	tokenizer·vocab·길이 호환성
EAGLE / EAGLE3	Target feature를 활용하는 draft	대상 모델용 checkpoint
MTP	모델 계열의 multi-token 모듈	native 지원 또는 지정 assistant
DSpark / DFlash / dynamic	특화 draft·병렬 제안·가변 예산	전용 recipe·sampling 조건 검증

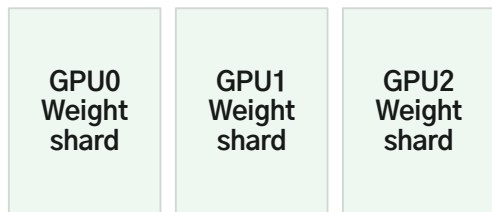
Chapter 6.

분산 실행과 모델별 실행 Recipe

TP · PP · DP

모델을 “들어가게 하기”와 요청 처리량을 “늘리기”를 구분합니다.

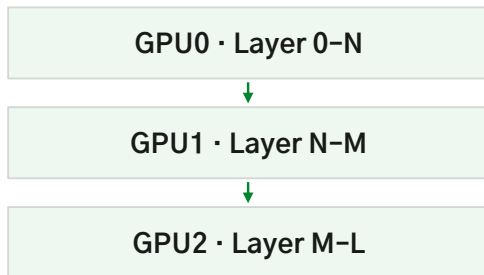
TP · Layer 내부



같은 요청의 연산을 결합

통신 증가

PP · Layer 그룹



pipeline bubble 고려

DP · 모델 Replica



요청 단위 독립 처리

GPU 수가 같아도 TP×DP 배치와 interconnect에 따라 지연·처리량·비용이 달라집니다.

TP · Tensor Parallelism

한 Layer의 연산을 여러 GPU가 나눠 계산하고, Layer마다 결과를 합칩니다.

정의와 동작

Layer의 weight 행렬을 GPU 수만큼 잘라 여러 GPU가 동시에 계산합니다.

Layer마다 all-reduce로 부분 결과를 합치므로 GPU 간 통신이 잦습니다.

모델이 GPU 1장에 올라가지 않을 때 가장 먼저 쓰는 수단입니다.

GPU Machine 배치

1 노드 · H100 80GB × 4 (NVLink)

GPU0
weight 1/4

GPU1
weight 1/4

GPU2
weight 1/4

GPU3
weight 1/4

Layer마다 all-reduce 동기화

언제 — 모델이 1장에 안 들어갈 때. TP=2/4/8을 한 노드 안에서만 두고 노드 경계는 넘지 않습니다.

TP는 NVLink로 묶인 한 노드 안에서만 키웁니다. 노드를 넘기면 all-reduce 지연이 처리량을 깎습니다.

PP · Pipeline Parallelism

모델을 Layer 구간으로 잘라 GPU가 순서대로 이어받아 계산합니다.

정의와 동작

모델을 Layer 구간(stage)으로 잘라 GPU에 순서대로 배치합니다.

stage 경계에서 활성값만 넘기므로 통신량이 TP보다 훨씬 작습니다.

요청이 stage를 차례로 지나 빈 구간(pipeline bubble)이 생깁니다.

언제 — 한 노드의 TP로도 모델이 안 들어갈 때. 노드를 넘겨 모델을 담는 용도로 씁니다.

GPU Machine 배치

노드 1 · GPU0-7 (TP8) · Layer 0-39

InfiniBand · 활성값만 전달

노드 2 · GPU0-7 (TP8) · Layer 40-79

micro-batch가 적으면 bubble 발생

PP는 노드를 넘겨 모델을 담는 수단입니다. bubble 탓에 지연보다 처리량 관점에서 씁니다.

DP · Data Parallelism (Replica)

같은 모델을 여러 번 복제해 요청을 나눠 처리합니다.

정의와 동작

모델 1벌이 올라간 GPU 묶음을 여러 개 복제해 요청을 나눠 받습니다.

replica 사이에 통신이 없어 처리량이 replica 수에 거의 비례합니다.

replica마다 weight와 KV cache를 따로 들고 있어 메모리를 더 씁니다.

언제 — 모델은 이미 들어가고 QPS·대기열이 문제일 때. 남은 GPU를 replica로 늘립니다.

GPU Machine 배치

1 노드 · H100 80GB × 8

요청 큐 → 라우팅

Replica 0 · GPU0-3 (TP4) · 요청 A·B

Replica 1 · GPU4-7 (TP4) · 요청 C·D

TP는 모델을 담고 DP는 트래픽을 담습니다. 같은 8장이면 TP8×DP1은 지연, TP4×DP2는 처리량 우선입니다.

GPU Machine별 배치 예시

먼저 TP로 모델을 노드 안에 담고, 남은 GPU를 DP로 복제하고, 노드를 넘어야 할 때만 PP를 더합니다.

모델 · 요구	GPU Machine	배치	이유
Llama 8B (FP16 ~16GB)	A100 80GB × 1	TP1 × DP1	1장에 그대로 올라갑니다. 처리량이 부족하면 DP로만 늘립니다.
Llama 70B (FP16 ~140GB)	H100 80GB × 4 (1노드, NVLink)	TP4 × DP1	weight와 KV가 1장을 넘어 노드 안 TP로 담습니다.
Llama 70B · 처리량 우선	H100 80GB × 8 (1노드, NVLink)	TP4 × DP2	같은 8장으로 동시 처리량 2배, 지연은 소폭 상승합니다.
Llama 405B (FP16 ~810GB)	H100 80GB × 8 × 2노드 (IB)	TP8 × PP2	노드 안은 TP8, 노드 사이는 통신량이 작은 PP로 잇습니다.

FP16 weight 기준 개략치이며 KV cache는 별도로 잡아야 합니다.

배치는 모델 크기가 먼저, 트래픽이 그다음입니다. 같은 GPU 수라도 조합에 따라 지연과 처리량이 갈립니다.

Dense 모델 Recipe

TP 증가는 Weight를 분산하지만 collective communication도 늘립니다.

01

1 · Fit 조건

가중치뿐 아니라 목표 context와 KV, graph 공간까지 들어가는 최소 구성을 찾습니다.

모델 fit → 운영 fit

02

2 · TP 후보

단일 요청 지연·메모리 headroom을 개선할 수 있지만 통신이 이득을 상쇄할 수 있습니다

NVLink / PCIe 확인

03

3 · DP 후보

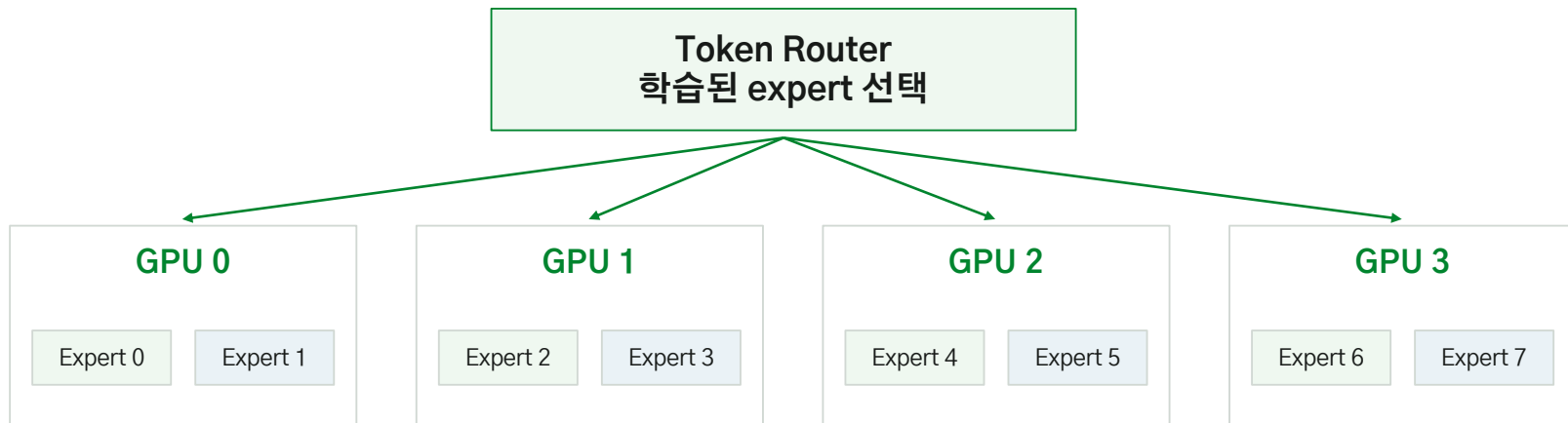
충분히 작은 replica 여러 개로 독립 요청을 나눕니다. Router와 prefix locality도 봅니다.

처리량 / GPU · 비용

“TP를 늘리면 항상 빨라진다” 대신, 같은 GPU 예산의 Pareto 결과로 선택합니다.

MoE 모델 Recipe

Active parameter 수가 작아도 모든 expert weight의 배치 문제는 남습니다.



All-to-all dispatch / combine · 쓸림과 interconnect 비용을 측정

EPLB (Expert Parallel Load Balancer) — 쓸린 expert를 복제·재배치해 GPU 부하를 고르게 맞춥니다

EPLB는 쓸림을 줄이는 후보입니다. Redundant expert가 쓰는 메모리와 재배치 비용도 포함합니다.

EPLB · Expert Parallel Load Balancer

vLLM이 expert별 부하를 모아 두고, 주기마다 expert의 복제본과 배치를 다시 정합니다.

정의와 동작

MoE는 expert마다 받는 토큰 수가 달라 일부 GPU만 바빠집니다.

vLLM은 forward마다 expert별 부하를 window로 누적합니다.

주기마다 쏠린 expert를 복제 (redundant expert)해 GPU에 다시 배치합니다.

언제 — MoE 모델을 EP로 돌리고 GPU 사용률 편차가 클 때

설정 — `--enable-eplb`로 켜고, `--eplb-config`로 `window_size · step_interval · num_redundant_experts` 조정

GPU Machine 배치

1 노드 · GPU0-3 · Expert Parallel

적용 전 — 인기 expert가 GPU0에 몰려 GPU2·3은 유향

적용 후 — 인기 expert 복제본을 GPU2·3에도 배치

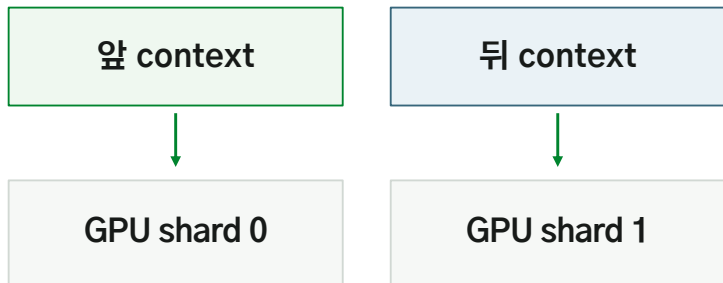
재배치마다 GPU 간 weight 이동 비용 발생

EPLB는 쏠림을 줄이는 대신 메모리와 재배치 비용을 씁니다. 실측으로 이득을 확인한 뒤 켵니다.

Context Parallelism

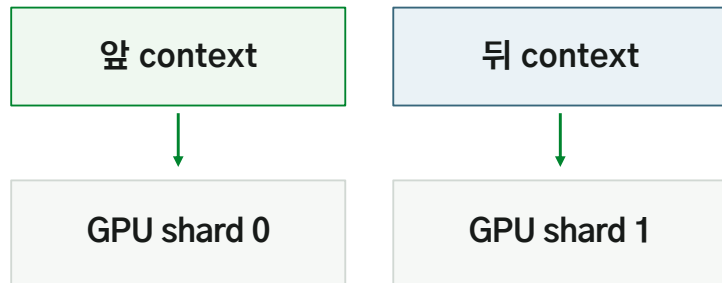
PCP와 DCP는 긴 context의 계산·KV 접근을 분산하는 선택지입니다.

PCP · Prefill Context Parallel



긴 prompt의 계산을 나눔

DCP · Decode Context Parallel

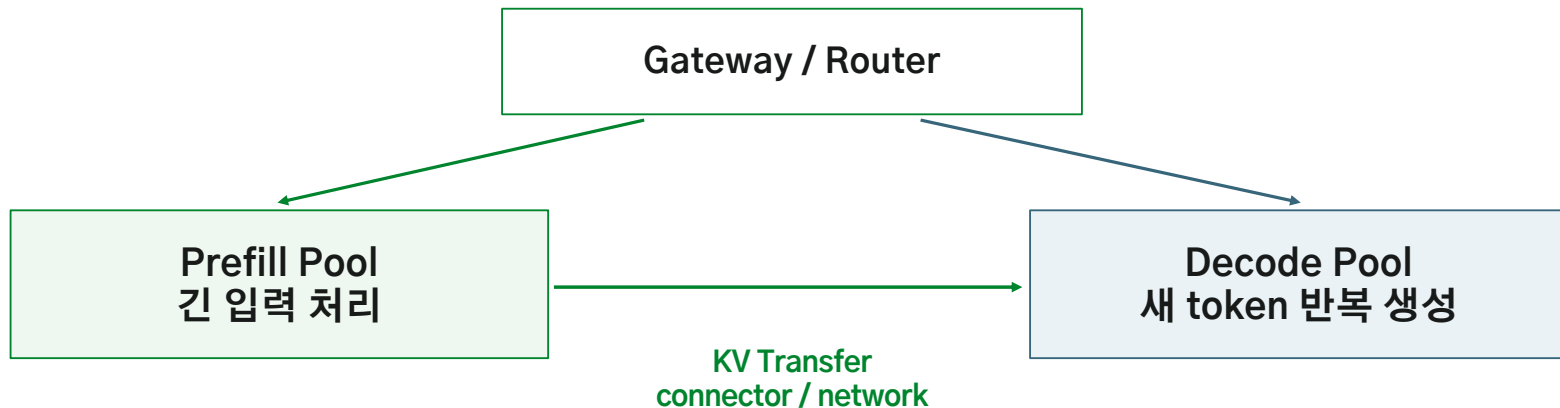


과거 KV를 나눠 읽고 결과 결합

TP의 단순 대체가 아닙니다. 지원 backend와 TP/CP 배수 제약, 통신 비용을 모델 Recipe로 검증합니다.

Prefill / Decode 분리

긴 prefill이 decode 지연을 흔드는 대규모 서비스에서 검토할 고급 구조입니다.



NIXL·Mooncake 등 connector의 지원 조합과 설치 조건을 확인합니다.

단일 엔진의 cache·scheduler 튜닝으로 해결되지 않을 때 검토

이득 = 간섭 감소·독립 튜닝 이점 - KV 전송·라우팅·추가 대기 비용. 공식 문서는 experimental로 분류합니다.

Offload와 Sleep

메모리 용량·재사용·유휴 자원 반환의 목적을 구분합니다.

기능	언제 유효한가	새로 생기는 비용
Weight offload	가중치가 HBM에 맞지 않을 때	CPU↔GPU 접근·전송
KV tiering / offload	긴 prefix를 계층 간 재사용할 때	lookup·transfer·eviction
Sleep mode	유휴·RL 단계 전환·모델 전환	wake-up·reload·cache 재준비
P/D separation	서로 다른 부하를 독립 확장할 때	KV 이동·network·운영 복잡도

용량 확대나 자원 효율의 개선을 steady-state decode TPS 향상으로 잘못 표현하지 않습니다.

LoRA Adapter 운영

원본의 AWQ/BnB+LoRA 실험을 원인 분리가 가능한 실험으로 바꿉니다.

1 실제 최대 rank에 맞춤

사용 adapter의 최대 rank가 64라면 불필요하게 256으로 예약하지 않습니다.

2 동시 활성 수를 분리

max_loras와 실제 요청의 adapter 혼합을 기록합니다.

3 교차 효과를 측정

정밀도 × rank × 동시성. 먼저 O2 기준선에서 한 변수씩 바꿉니다.

실행 예시 / GPU 실측은 별도

```
vllm serve "$BASE_MODEL" \  
  --enable-lora \  
  --lora-modules domain="$LORA_PATH" \  
  --max-lora-rank 64 \  
  --max-loras 4 \  
  -O2
```

큰 rank·TP·sequence 조건에서는
fully-sharded-loras도 별도 A/B

원본의 특정 TPS 표는 당시 환경의 사례입니다. 최신 조합의 성능 순위를 뜻하지 않습니다.

구조화 출력과 E2E 지연

GPU가 출력한 첫 토큰과 사용자가 처음 보는 답변은 같지 않을 수 있습니다.



엔진 TTFT

첫 model token이 반환되기까지의 시간. Reasoning token이 먼저 나올 수도 있습니다.

사용자 체감 지연

첫 visible answer 또는 완성된 tool-call 인자가 준비되기까지. Parser와 실제 도구 시간도 구분합니다.

JSON schema 통과는 인자 내용의 정답을 보장하지 않습니다. 형식·의미·업무 성공을 따로 검증합니다.

Schema 유효성·tool 인자·parser 오류와 첫 visible-answer latency를 함께 측정합니다. 재시도 감소도 업무 처리량에 반영합니다.

모델별 Runtime 개선 이력

기능을 단순히 켜는 수준을 넘어 kernel·cache·sampling·통신을 함께 개선합니다.

모델 / 시점	vLLM의 구현 노력	줄이려는 병목
Kimi K3 · v0.27-0.29	KDA·AttnRes kernel, DCP, NVFP4 지원	hybrid attention · KV · expert
DeepSeek V4 · 최근 릴리스	sparse MLA·fused kernel·DSpark 검증 경로	sparse attention · launch · draft
Qwen3.8-Flash-Next · v0.29	BF16/FP8/NVFP4·MTP 지원	큰 weight · hybrid state · decode
MRV2 · v0.29	graph memory profiling·샘플링 분산	HBM 예산 · logits · 동기화

Kernel 단위 가속률을 전체 서비스 가속률로 합산하지 않습니다. 동일 조건 E2E 재측정이 필요합니다.